

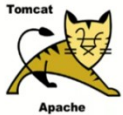
Application Server VS Servlet Container

Application server



Le applicazioni Java EE girano su dei server java;
Un Application server supporta tutto lo stack di api di java EE
mentre un servlet container supporta solo le servlet e le pagine JSP

Servlet container



Una servlet è una classe Java che estende una class HttpServlet (estesa da GenericServlet);
con un'annotazione @WebServlet. Possiamo evitare l'annotazione se aggiungiamo un nuovo file di tipo web → standard deployment description (web.xml)

Tool che gestisce le dipendenze E' uno strumento che vi permette di:

Usate build tool e
gestori di dipendenze



Maven



- Gestire le dipendenze di un progetto
- Compilare le classi e impacchettarle (.jar, .war)
- Ulteriori plugin

Pom.xml (project object model) è un file che contiene tutta la configurazione Maven riguardante il Vostro progetto.

Con Maven abbiamo la possibilità di integrare delle classi Java dentro alle classi con l'annotazione, a creare gli oggetti e iniettarli nella classe ci pensa appunto Maven; una volta configurato opportunamente il POM; integrando al suo interno eventuali librerie (plugin) necessari.

CDI Context - Dependency - Injection



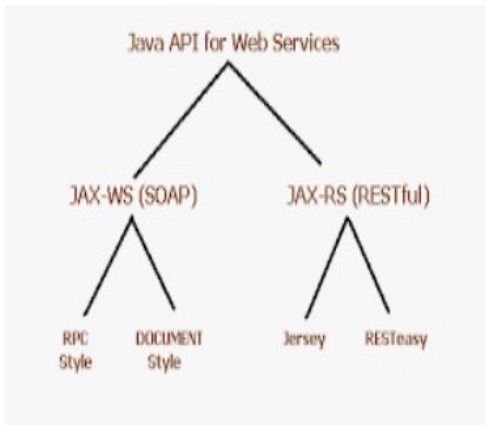
- Context
dove vive un oggetto, se durante la singola richiesta http, se vive durante tutta la session, se vive per la durata dell'applicazione e così via
- Dependency
E' una tecnica che consente ad una classe di definire le sue

dipendenze, nel nostro da altre classi e poi qualcun altro si preoccuperà di creare gli oggetti che gli servono e di farglieli trovare già pronti quando servono

- Injection

è proprio la capacità di JEE di creare in autonomia gli oggetti che occorrono e iniettarli nella classe che ne fa richiesta

Servizi REST con JAX-RS



JAX-RS è lo standard Java EE che fornisce le API sia per costruire un servizio REST che per creare applicazioni che facciano da Client a questi servizi REST.

L'implementazione di riferimento, contenuta in Glassfish è Jersey.

I file `ApplicationConfig.java` e il file `Resource`

1. `ApplicationConfig.java`

Serve per configurare la web service; non va modificata, automaticamente verranno aggiunte le classe resource, classi che hanno all'interno annotazioni `@path` con la quale la classe `ApplicationConfig` riesce a ricostruire l'url su quale andremo a chiamare il servizio REST.

L'unica cosa che ha senso modificare è l'`applicationPath` che è la prima parte di url; che viene scritta dopo il package, ma viene digitata prima dei path dei servizi REST aggiunti con l'`@Path`.

2. `UtentiResource.java`

Rappresenta una richiesta http, infatti troverai annotazioni come `@Path` e `@Get`; che indicano una parte di path dell'url e la richiesta in Get di una pagina web.

JPA

Java Persistence API

JPA = Java Persistence API
ORM = object-relational mapping
RMDBS ↔ OO



È la specifica di Java EE che si occupa di fornire le implementazioni per l'ORM.

L'ORM indica una tecnologia che permette di trasformare i dati contenuti in un qualsiasi Database relazionale (quelle che usano il linguaggio SQL) in oggetti di un dato linguaggio di programmazione.

Entity Class

E' una classe java che corrisponde ad una tabella del database

Transazione

Un apposito modulo del DBMS, denominato Transaction Manager (TM), ha il compito di coordinare tutte le operazioni affinché le transazioni soddisfino le proprietà ACID di Atomicità, Consistenza, Isolamento e Durabilità. Tali proprietà hanno lo scopo di assicurare che lo stato del database sia sempre consistente, indipendentemente dalle operazioni eseguite e dal loro esito.

- A Atomicità
- C Consistenza
- I Isolamento
- D Durabilità

Queries

Queries

- Java Persistence Query Language (JPQL)
- Criteria API
- Query native

Nella JPQL anziché ragionare su tabelle e colonne si ragiona sulle entity class
Con le criteria API si utilizzano dei metodi
E poi si possono lanciare le sql del database direttamente da java

JPA - Cascade sulle relazioni

Cascade

```
class Utente {  
    @OneToMany(cascade = CascadeType.PERSIST)  
    private List<Libro> libri;  
}
```

Enum <u>CascadeType</u>	Metodi di EntityManager
MERGE	merge(T entity)
PERSIST	persist(Object entity)
REMOVE	remove(Object entity)
REFRESH	refresh(Object entity)
DETACH	detach(Object entity)
ALL	

Il compito del Cascade è quello di propagare una sua azione nelle classi coinvolte nella relazione.

Managed IL COMMIT è l'autocommit su tutta la transazione

Detach IL COMMIT è manuale

L'orphan remove elimina tutte le righe di una tabella che al momento non hanno relazione attive.

JPA - FetchType delle relazioni e serializzazione

- Lazy
L'entity manager caricherà dal database i dati per l'entità solo quando servono; ovvero, quando verrà richiamato un metodo della entity manager su quella entità. (problemi nei bean di sessione perché sono serializzati).

- Eager

L'entity manager caricherà sempre tutti i dati dal database per l'entità

JAXB = Java Architecture for XML Binding

```
-----
Building jaxbdemo 1.0
-----
--- exec-maven-plugin:1.2.1:exec (default-cli) @ jaxbdemo ---
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<utente>
  <email>pippo@pippo.com</email>
  <libri>
    <autore>Isaac Asimov</autore>
    <titolo>Io, Robot</titolo>
  </libri>
  <libri>
    <autore>Michael Crichton</autore>
    <titolo>Preda</titolo>
  </libri>
  <nome>pippo</nome>
</utente>
```

JAVA → XML (marshalling)
XML → Java (unmarshalling)

L'annotazione @XmlRootElement

In pratica stiamo dicendo che la nostra classe deve corrispondere ad un elemento radice di un file Xml

JNDI Java naming and directory interface

E' uno standard del JEE usato per reperire delle risorse che sono sul nostro server (GLASSFISH) mediante una stringa che contiene il nome della risorsa. Cosa intendiamo come risorsa, il più delle volte saranno connessioni verso un database ma potenzialmente potrebbe essere qualsiasi oggetto.

JDBC Connection Pool

Crea un numero di connessioni pronte, Le connessioni create (pool) saranno prese in prestito all'occorrenza e rilasciate quando non serviranno più.

N.B. Ricorda che glassfish deve avere il driver di connessione al database in uso per creare queste connessioni

Java Server Faces

Pattern MVC

Lo standard JSF segue il pattern MVC; MVC è lo standard che separa il codice che rappresenta il vostro modello dati (Model), dal layout (pagine web); dalla logica dell'applicazione.

Annotazione @Named

E' necessaria per poter vedere un'istanza di una classe nella pagina web di tipo JSF.

Facelets offre molto di più di una pagina jsp (oramai è in disuso); le pagine jsf hanno come estensione xhtml.

Resources per la personalizzazione di una pagina JSF



The screenshot displays a web IDE interface. On the left, a file explorer shows a project structure with folders like 'Web Pages', 'WEB-INF', 'resources', 'css', 'fonts', 'img', 'js' and a file 'index.xhtml'. The main area on the right shows the XML content of 'index.xhtml'.

```
<?xml version='1.0' encoding='UTF-8' ?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:h="http://xmlns.jcp.org/jsf/html">
  <h:head>
    <title>Facelet Title</title>
    <h:outputStylesheet library="css" name="style.css" />
    <h:outputScript library="js" name="script.js" />
  </h:head>
  <h:body>
    <p>Hello from Facelets</p>
    <div class="container"></div>
    <h:graphicImage library="img" name="topo-gigio.jpg" />
  </h:body>
</html>
```

Index.xhtml